MAI605 – Robotic Systems (ROS2)

Technical Report: Industrial Pick-Scan-Place Robot Cell

Program:

Master of Science in Artificial Intelligence

Course:

MAI605 – Robotic Systems (ROS2)

Instructor:

Prof. Omar Osama Shalash

Submitted by:

Maryam Ali | ID 202512707

Maitha Alghfeli | ID: 202512976

Date: April 30, 2026

1. Project Overview

This project implements an industrial-grade pick–scan–place robotic cell built entirely within the ROS 2 ecosystem. The central idea is straightforward: a 6-DOF robot arm equipped with a gripper picks a QR-tagged object from a fixed fixture, lifts it clear of the workspace, carries it to a scan station, reads the QR code, determines the correct destination bin from the encoded payload, and places the object accordingly before returning to its home configuration.
The toolchain driving this system includes MoveIt 2 for motion planning, Gazebo Classic for simulation, ros2_control for low-level trajectory execution, and zbar_ros for QR perception. A lightweight coordinator state machine ties these elements together, managing the sequencing and handling error conditions along the way. The result is a tightly integrated demonstration of how modern robotics middleware can be combined to realize a complete, task-level autonomous workflow.

2. Project Scope and Objectives

The workspace is organized around a clear separation of concerns, where each ROS 2 package owns exactly one functional layer of the system. This architectural discipline is not just a software engineering preference; it directly reflects the robust structure of industrial automation software.
The packages and their respective responsibilities are as follows:

Package	Responsibility
robot_description	Robot model, link geometry, inertials, RViz support
moveit_config	SRDF, planner configuration, kinematics plugin, controller mapping
workflow_interfaces	Shared BinTarget.msg and ExecuteStage.srv
manipulation	Main MoveIt-based stage execution node
decision_logic	QR-to-bin routing and target pose publication
simulation	Gazebo world, models, and object attachment helper
bringup	Full-system launch and workflow coordinator

Several key assumptions define the scope of this project. The workspace remains static, the pick location is fixed, and scan and bin regions are predefined. Online pose estimation is not performed, force feedback is unavailable, and grasping is managed by object repositioning in simulation instead of contact-rich physics. These simplifications are intentional to maintain focus on system integration and workflow orchestration, which provide the primary pedagogical value.

3. Workspace and Package Architecture

The repository is structured as a standard ROS 2 workspace. At runtime, the system is data-driven in a clean, pipeline fashion. A camera image is decoded into a string payload, the decision node translates that payload into a BinTarget message, the coordinator waits for the appropriate stage transition and calls the manipulation service, and the manipulation node plans and executes the requested stage. Meanwhile, an attachment helper keeps the carried object visually aligned with tool0 throughout the grasp phase.

This kind of clean data flow perception feeds decision logic, which feeds manipulation, which is a pattern worth internalizing. When the interfaces between modules are well-defined, the individual components become independently testable and replaceable.

4. Robot Mechanical Structure

4.1 Topology

The robot is a serial manipulator with six revolute arm joints and one prismatic gripper joint. Two additional fixed frames — tool0 and camera_link — serve as the end-effector and the sensor reference points, respectively. The arm chain in the SRDF runs as:
base_link → shoulder_link → upper_arm_link → forearm_link → wrist_pitch_link → wrist_yaw_link → wrist_roll_link → gripper_base_link → tool0
The arm planning group spans from base_link to tool0, while the gripper planning group contains the single prismatic joint.

4.2 Links, Geometry, and Inertial Modeling

The URDF model uses only simple primitive geometry — boxes and cylinders. This keeps the model efficient for collision checking without sacrificing kinematic accuracy. Inertia tensors are generated through two reusable Xacro macros: box_inertial and cylinder_inertial. The physical links are summarized below.

Link	Shape	Dimensions	Mass
base_link	Cylinder	radius 0.12 m, length 0.70 m	18.0 kg
shoulder_link	Box	0.14 × 0.16 × 0.16 m	4.0 kg
upper_arm_link	Box	0.36 × 0.10 × 0.12 m	5.0 kg
forearm_link	Box	0.32 × 0.09 × 0.10 m	3.8 kg
wrist_pitch_link	Box	0.16 × 0.08 × 0.08 m	1.4 kg
wrist_yaw_link	Cylinder	radius 0.045 m, length 0.12 m	1.0 kg
wrist_roll_link	Cylinder	radius 0.04 m, length 0.10 m	0.9 kg
gripper_base_link	Box assy	main body 0.12 × 0.10 × 0.06 m	0.8 kg
gripper_slider_link	Box	0.10 × 0.02 × 0.07 m	0.2 kg
camera_link	Box	0.04 × 0.04 × 0.04 m	0.1 kg

A few modeling choices are worth highlighting. Collision bodies are present on every link, which is key for reliable motion planning. The tool0 frame is a massless fixed reference link used as the planning tip; it plays no role in dynamics yet is central to all Cartesian planning operations. The gripper is simplified to a single prismatic slider rather than a full two-finger mechanism, and the wrist-mounted camera is modeled but the main QR pipeline defaults to the fixed scan-station camera in the Gazebo world.

4.3 Joints and Degrees of Freedom

The robot has seven controllable joints in total — six for the arm and one for the gripper. The table below summarizes their types, parent-child relationships, origin offsets, axes of motion, and position limits.

Joint	Type	Parent → Child	Origin (m)	Axis	Limits
shoulder_pan_joint	Revolute	base_link → shoulder_link	(0, 0, 0.80)	(0,0,1)	[-π, π]
shoulder_lift_joint	Revolute	shoulder_link → upper_arm_link	(0.08, 0, 0)	(0,1,0)	[-1.8, 1.6]
elbow_joint	Revolute	upper_arm_link → forearm_link	(0.36, 0, 0)	(0,1,0)	[-2.1, 2.1]
wrist_pitch_joint	Revolute	forearm_link → wrist_pitch_link	(0.32, 0, 0)	(0,1,0)	[-2.4, 2.4]
wrist_yaw_joint	Revolute	wrist_pitch_link → wrist_yaw_link	(0.16, 0, 0)	(0,0,1)	[-π, π]
wrist_roll_joint	Revolute	wrist_yaw_link → wrist_roll_link	(0.12, 0, 0)	(1,0,0)	[-π, π]
gripper_joint	Prismatic	gripper_base → gripper_slider	(0.03, 0.04, 0)	(0,−1,0)	[0.0, 0.04] m

Fixed joints complete the mechanical model. The tool_mount_joint connects wrist_roll_link to gripper_base_link at an offset of (0.10, 0, 0), the tool0_joint connects gripper_base_link to tool0 at (0.14, 0, 0), and the camera_mount_joint places the camera at (0.08, 0, 0.08) relative to the gripper base.

4.4 Structural Interpretation

The joint axis sequence — (z, y, y, y, z, x) — gives this robot a recognizable architecture: one base yaw joint, a three-joint pitch-based chain for positioning the elbow and wrist in a vertical plane, and a three-axis wrist for direction control. Together these provide a full 6D end-effector pose in principle, subject to joint limits, collision constraints, and solver convergence.
The nominal serial-chain length along the local x-directions sums to 0.08 + 0.36 + 0.32 + 0.16 + 0.12 + 0.10 + 0.14 = 1.28 m. This figure represents the theoretical maximum reach of the unfolded chain, not a practical workspace radius. The effective reachable volume is considerably smaller once joint limits, self-collision, table and bin collisions, orientation constraints, and planner filtering are all taken into account.

5. Semantic Model, Planning Groups, and Named States

The SRDF encodes the higher-level semantic structure of the robot above and beyond what the URDF describes kinematically. It defines a virtual joint anchoring the robot to the world frame, the arm planning group as a kinematic chain from base_link to tool0, the gripper planning group over the single prismatic joint, and the gripper_ee end-effector.
Named arm states defined in the SRDF — home, pre_grasp, grasp, lift, transit, scan_pose, bin_a, bin_b, and bin_c — serve as high-level waypoints that the manipulation node can request by name, without requiring online IK to compute the corresponding joint configuration. The gripper has two named states: gripper_open and gripper_closed. This is a very practical design pattern: for commonly visited configurations, it is far more reliable to store known-good joint values than to re-solve IK every time.

6. System Functionality and End-to-End Workflow

6.1 Functional Objective

The cell executes the following repeatable sequence on each cycle:

- Reset the QR object to the pick fixture
- Move the robot to the pick-ready pre_grasp pose
- Close the gripper and simulate object attachment
- Lift the object to a safe transit height
- Move to the scan region and hold the object for perception
- Decode the QR payload from the scan camera
- Map the payload to a target bin and planning pose
- Move the arm above the selected bin
- Release the object into the bin
- Retreat and return to the home configuration

6.2 Coordinator State Machine

The coordinator node publishes its current state on /coordinator/state and orchestrates all manipulation service calls. The nominal state progression is:

IDLE → MOVE_TO_PICK → GRASP → LIFT → MOVE_TO_SCAN → WAIT_FOR_QR → DECIDE_BIN → MOVE_TO_PLACE → RELEASE → RETURN_HOME → IDLE.

Beyond sequencing, the coordinator handles optional auto-start after a configurable delay, object reset before each cycle begins, timeout management while waiting for a valid QR scan decision, enabling and disabling the attachment helper at the right moments, and suppressing duplicate QR and bin-target log messages. These details are precisely what separates a working demo from a system that would hold up under repeated operation.

6.3 Manipulation Stage Semantics

The manipulation node exposes a single service, /manipulation/execute_stage, which accepts a stage name string and returns a success flag with a message. The key stages and their internal logic are worth examining:

- move_to_pick opens the gripper, refreshes the object in the planning scene, and reaches a pre-grasp pose. grasp first attempts the named grasp state, then falls back to a short Cartesian descent, then to an object-offset pose target, before closing the gripper and attaching the object in the planning scene.
- lift rises through either a named pose, an object-offset target, or a vertical Cartesian fallback. move_to_scan passes through transit and scan waypoints, holding the object for perception.
- move_to_place ascends to a clearance height, translates toward the target bin via a Cartesian path, and falls back to named place poses if needed. release descends, opens the gripper, detaches the object, places it into the bin, and retreats.
- return_home tries staged retreat strategies and accepts a safe fallback if the exact home pose is blocked by collision.

This layered fallback structure is a deliberate engineering decision. In real industrial cells, planners fail. Building explicit fallback chains rather than crashing on the first IK failure is what makes a system usable.

7. Perception Pipeline

The perception stack supports two operating modes: a deterministic demo mode that streams a synthetic QR image, and a camera mode that processes images from the simulated scan-station camera. For demonstration and testing, the mock stream is highly valuable because it decouples perception failures from planning and execution failures — two very different problems.

7.1 Mock QR Stream

mock_qr_stream.py generates a QR image encoding a configurable payload (defaulting to RED) and publishes it as a sensor_msgs/Image at 2 Hz on /demo_qr/image_raw. The image is 480 × 480 pixels and uses the camera_optical_frame coordinate frame.

7.2 Barcode Reader and Bridge

zbar_ros handles the actual QR decoding and publishes raw text on /barcode. A custom bridge node subscribes to this topic, converts the text into uppercase, rejects unsupported payloads, deduplicates repeated messages, and republishes clean output to /perception/decoded_qr. The accepted default payloads are RED, BLUE, and GREEN. The deduplication step is important in practice — without it, a static QR code would flood downstream nodes with redundant BinTarget messages on every camera frame.

7.3 Scan Cameras

Two camera definitions exist in the project: a wrist-mounted camera defined in the URDF (camera_link and camera_optical_frame), and a fixed scan-station camera defined in the Gazebo world. The default pipeline uses the external scan-station camera, which provides a clean, unmoving view of the scan region. The wrist camera remains available in the model as a foundation for future extensions, such as in-hand object inspection or pose refinement.

8. Decision Logic

The decision node is conceptually simple but functionally important: it maps normalized QR strings to logical bin identities and corresponding placement poses. The default routing is:

- **RED → bin_a at (0.62, 0.55, 1.18)**
- **BLUE → bin_b at (0.82, 0.55, 1.18)**
- **GREEN → bin_c at (1.02, 0.55, 1.18)**

All three bins share the same end-effector orientation, encoded as the quaternion (0, −0.7071068, 0, 0.7071068), which corresponds to a tool-down approach. The BinTarget message carries the QR text, the bin identifier, and a stamped pose. This clean interface allows the manipulation node to remain entirely agnostic to perception and routing logic — it simply receives a goal pose and executes.

9. Gazebo Simulation Environment

The Gazebo world — pick_scan_place.world — models a compact sorting cell. It contains a pick table with a support fixture, a scan pedestal with a camera rig, a bin rack holding three sorting bins, and the QR-tagged object. The key world poses are summarized below.

Element	World Pose	Notes
Pick table	(0.72, 0.00, 0)	Tabletop at z = 0.75 m
Scan pedestal	(0.40, −0.48, 0)	—
Bin rack	(0.82, 0.55, 0)	—
Bin A	(0.62, 0.55, 0.75)	—
Bin B	(0.82, 0.55, 0.75)	—
Bin C	(1.02, 0.55, 0.75)	—
QR object	(0.72, 0.00, 0.825)	Gravity disabled

A few modeling simplifications are worth noting. The pick table visual includes table legs for realism, but the MoveIt planning scene models only the tabletop footprint — a flat box at the appropriate height. This keeps collision checking simple and avoids spurious planning failures caused by small gaps between modeled geometry. The QR object has gravity disabled and is repositioned explicitly during reset and release operations. The sorting bins are static visual boxes with no physics simulation.
Grasping is simulated by continuously updating the object pose to track tool0 through TF. This gives deterministic, visually coherent grasp behavior at the cost of physical realism. For a teaching platform, this trade-off is entirely appropriate.

10. Motion Planning and Control

10.1 MoveIt 2 Configuration

The motion planning stack uses the OMPL interface with geometric::RRTConnect as the default arm planner and geometric::RRTstar available as an alternative for quality-optimized paths. Standard request adapters handle workspace bounds, start state, constraints, and time parameterization. Trajectory execution settings allow a 1.2× duration scaling, a 0.5 s goal duration margin, and a 0.01 rad/m start tolerance.

10.2 Kinematics Solver

The arm group uses the KDL numerical solver (kdl_kinematics_plugin/KDLKinematicsPlugin) with a search resolution of 0.005, a timeout of 0.1 s, and 3 attempts per IK call. This means inverse kinematics is solved numerically, not analytically. The consequences of this choice are discussed in detail in Section 12.

10.3 ros2_control and Controllers

The control stack includes a joint_state_broadcaster, an arm JointTrajectoryController, and a gripper JointTrajectoryController. The controller manager runs at 100 Hz, joint states are published at 100 Hz, and the action monitor rate is 20 Hz. The arm controller drives all six revolute joints; the gripper controller drives gripper_joint independently.

10.4 Planning and Execution Strategy

Rather than relying on a single motion type, the manipulation node mixes named joint-space targets, full 6D pose targets, position-only targets, and Cartesian path segments, selecting the form best suited to each stage of the task. Key execution parameters configured in poses.yaml include a planning time of 4.0 s, 5

planning attempts per goal, up to 2 maximum retries, velocity scaling of 0.38, and acceleration scaling of 0.28. Cartesian paths use a step size of 0.01 m with a jump threshold of 0.0.
This mixed strategy shows a genuine insight: some motions are fundamentally easier to express in joint space, while others are best described as Cartesian retreats or descents. Forcing every motion into one representation leads to brittle behavior.

11. Forward Kinematics

11.1 Role in the System

Forward kinematics maps a given set of joint coordinates to the corresponding Cartesian pose of any link in the chain. Within this project, FK is continuously used by robot_state_publisher to broadcast TF transforms, by MoveIt for pose evaluation and collision checking, and by the attachment helper to track the tool0 pose and update the simulated object position accordingly.
Let the arm joint vector be defined as:

$$q = [q1, q2, q3, q4, q5, q6]\ ^T$$

where q1 through q6 correspond to shoulder_pan_joint, shoulder_lift_joint, elbow_joint, wrist_pitch_joint, wrist_yaw_joint, and wrist_roll_joint respectively. The gripper coordinate qg represents gripper_joint.

11.2 Serial-Chain FK to tool0

Reading directly from the URDF joint origins and axes, the homogeneous transform from world to tool0 can be written as a product of elementary translations and rotations:

$$T_world_tool0(q) = Tz(0.80) \cdot Rz(q1) \cdot Tx(0.08) \cdot Ry(q2) \cdot Tx(0.36) \cdot Ry(q3) \cdot Tx(0.32) \cdot Ry(q4) \cdot Tx(0.16) \cdot Rz(q5) \cdot Tx(0.12) \cdot Rx(q6) \cdot Tx(0.24)$$

The last Tx(0.24) combines the two fixed tool offsets — Tx(0.10) from wrist_roll_link to gripper_base_link and Tx(0.14) from gripper_base_link to tool0 — since neither depends on any joint variable. Reading this expression physically: q1 rotates the entire arm about the vertical base axis; q2, q3, and q4 primarily control the reach and height of the wrist through a pitch-based chain; q5 and q6 adjust the orientation of the tool near the target. Since tool0 is fixed relative to the gripper base, the gripper slider position has no effect on the tool0 transform.

11.3 Gripper and Camera Forward Kinematics

The gripper slider transform is well-defined even though the slider is not the planning tip:

$$T_gripper_base_gripper_slider(qg) = Tx(0.03) \cdot Ty(0.04 - qg)$$

With qg ranging from 0.0 (closed) to 0.04 m (open). The wrist camera frames are:

$$T_gripper_base_camera = Tx(0.08) \cdot Tz(0.08)$$
$$T_camera_camera_optical = Rx(-\pi/2) \cdot Rz(-\pi/2)$$

This rotation produces the standard optical frame convention — x right, y down, z into the scene — required by Gazebo ROS camera plugins.

11.4 Differential Kinematics

For local Cartesian motion, the standard first-order relationship is x_dot = J(q) · q_dot, where J(q) is the geometric Jacobian of the serial chain. This Jacobian is not manually implemented in this codebase; MoveIt and KDL compute it internally when solving IK and validating Cartesian path segments.

Knowing its role is important, however, because it explains why the robot may show velocity or force amplification near certain joint configurations — a phenomenon relevant to singularity assessment.

12. Inverse Kinematics

12.1 IK Role in the Project

Inverse kinematics maps a desired end-effector pose back into the joint coordinates needed to achieve it. In this system, IK is invoked whenever the manipulation node commands a full pose target, a position target, a world pose above the scan station or a bin, or any Cartesian path whose intermediate waypoints must remain kinematically valid. Named states, by contrast, bypass IK entirely — the joint values are simply read from the SRDF and executed directly.

12.2 The KDL Numerical Solver

The arm uses KDLKinematicsPlugin with a search resolution of 0.005, a 0.1 s timeout, and 3 attempts. This is a numerical solver, not an analytical one. The practical implications are notable: IK success depends on the starting joint configuration, how close the requested pose is to the boundary of the reachable workspace, whether any joint limit or collision constraint is active, and whether the solver converges within the allowed time. Near kinematic singularities — configurations where the system Jacobian loses rank — KDL will struggle, and even a nominally reachable pose may fail to solve reliably.

12.3 IK Usage Layers in Manipulation

The manipulation node uses IK in several complementary ways. setPoseTarget issues a full 6D constraint, the most specific and most likely to fail. setPositionTarget relaxes orientation and asks only for an (x, y, z) position, serving as a natural fallback when a specific orientation is difficult to achieve. computeCartesianPath handles short translational or vertical motions, with success measured by the achieved path fraction. When all Cartesian and pose-based procedures fail, the system can revert to known-good SRDF joint states as a final fallback.

12.4 IK During the Workflow

Each stage has a defined fallback hierarchy. During move_to_pick, the arm first attempts the named pre_grasp state, then falls back to an object-offset pose target. During grasp, it tries the named grasp state, then performs a short Cartesian descent, then targets an offset pose. During move_to_scan, it tries to use transit waypoints, followed by a world pose target, a named scan pose, or a position-only fallback. During move_to_place, the arm first rises vertically, then translates toward the target bin using Cartesian motion, before falling back to pose, position, or named place states as needed.

12.5 Common Causes of IK and Planning Failure

Understanding why IK fails is as important as knowing how to use it. The most common failure modes in this architecture are:

- The requested pose lies outside the reachable workspace.
- The pose is geometrically reachable but violates a joint limit or collides with environment geometry.
- The numerical solver times out before converging.
- The orientation constraint is too strict given the current joint configuration, or the planner's assumed start state is stale or inconsistent with the actual robot state.

12.6 Robustness Measures Implemented

The codebase addresses IK fragility through a combination of strategies: multiple planning retries, position-only fallback after pose-target failure, Cartesian fallback for vertical segments, named-state fallback for common waypoints, acceptance of a safe fallback posture after release if exact home is blocked, and careful alignment of the planning scene geometry with the Gazebo world. These measures combined make the system substantially more robust than a naive single-attempt approach.

13. Planning Scene and Collision Modeling

The manipulation node seeds a simplified planning scene that includes the pick_table, scan_table, bin_a, bin_b, bin_c, and the qr_object. Several deliberate simplifications improve planning stability. The pick table is modeled as the tabletop surface only — a thin box — rather than as a floor-to-surface solid. The scan pedestal is represented as a single box obstacle regardless of its visual complexity. Each sorting bin is a simple box. These approximating techniques lower the complexity of the collision-checking computation yet without significantly jeopardizing safety, provided the planned paths stay clear of the actual geometry.

One dynamic element of the planning scene is the QR object itself: it begins as a freestanding world collision object, transitions to an attached collision object on the robot when grasped, and returns to a world object upon release. Keeping the planning scene synchronized with this state transition is essential for correct collision avoidance throughout the cycle.

14. Simulated Attachment and Release

Physical grasping in simulation typically requires either contact-rich dynamics (computationally expensive and prone toward instability) or some form of kinematic coupling. This project takes the latter approach. Once the gripper closes, the manipulation node attaches the object in the MoveIt planning scene, and a dedicated simulation helper node begins continuously updating the Gazebo object pose to follow tool0 at 20 Hz.

The attachment helper looks up the world-to-tool0 transform via TF, rotates the configured attach offset by the tool quaternion, and writes the resulting object pose to Gazebo through /gazebo/set_entity_state. It exposes two services — /simulation/set_attachment for enabling and disabling object following, and /simulation/reset_object for returning the object to its pick fixture between cycles. The result is deterministic and visually coherent, even if it is not a physically realistic grasp model.

15. Topics, Services, and Actions

15.1 Published Topics

Topic	Message Type	Description
/barcode	std_msgs/String	Raw zbar_ros QR output
/perception/decoded_qr	std_msgs/String	Normalized QR payload
/decision_logic/bin_target	workflow_interfaces/BinTarget	Selected bin ID and target pose
/coordinator/state	std_msgs/String	Coordinator state machine status
/manipulation/status	std_msgs/String	Stage-level execution status
/demo_qr/image_raw	sensor_msgs/Image	Synthetic QR image stream
/scan_station_camera/image_raw	sensor_msgs/Image	Simulated scan camera image
/wrist_camera/image_raw	sensor_msgs/Image	Wrist camera image

15.2 Services

Service	Type	Description
/manipulation/execute_stage	workflow_interfaces/ExecuteStage	Execute one manipulation stage
/simulation/set_attachment	std_srvs/SetBool	Enable/disable object following
/simulation/reset_object	std_srvs/Trigger	Reset object to pick station
/coordinator/start_cycle	std_srvs/Trigger	Manually start workflow cycle

15.3 Actions

Two action servers drive hardware-level trajectory execution:

- /arm_controller/follow_joint_trajectory — arm trajectory execution
- /gripper_controller/follow_joint_trajectory — gripper trajectory execution

16. Software Behavior by Package

Each package in the workspace has a tightly defined role, and it is worth summarizing these briefly.

1. robot_description defines the kinematic chain, visual and collision geometry, inertial parameters, and sensor frames. It also adds ros2_control joint interfaces when Gazebo control is enabled and anchors the robot to the world through a fixed world_to_base_joint in Gazebo mode.
2. moveit_config supplies the SRDF groups and named states, connects MoveIt to the KDL solver and OMPL planners, and binds MoveIt to the arm and gripper controllers.
3. perception generates the mock QR stream for deterministic testing, runs zbar_ros over the selected image topic, and republishes normalized detections on the application-level topic.
4. decision_logic maps QR payloads to bin identities and publishes the logical target pose used for placement.
5. manipulation owns stage execution. It builds and retains the planning scene, performs MoveIt planning, manages controller fallbacks, handles Cartesian motion, and coordinates object attachment and release.
6. simulation launches Gazebo, stores the environment models, and runs the object-following helper to keep the simulated object synchronized with the tool during grasp.
7. bringup starts the full application stack — robot, Gazebo, controllers, MoveIt, perception, decision logic, manipulation, and coordinator — with carefully timed launch sequencing to ensure that controllers and action servers are ready before the workflow begins.

17. Challenges and Limitations

Building a complete robotic operation pipeline within a single academic project inevitably involves balances between realism, complexity, and scope. The complications encountered during development fall into two categories: deliberate simplifications that were made to keep the project focused, and genuine technical restrictions that would need to be addressed in any production deployment. Both are worth documenting clearly, as understanding the gap between a working demonstration and a deployable system is itself an important engineering skill.

- **No online object or bin localization.** All pick and placement positions are fixed and pre-measured in the URDF and world files. The system has no mechanism to detect or adapt to any variation in object or bin position at runtime.
- **Simulated grasping only.** Object attachment is achieved by continuously updating the Gazebo object pose to follow tool0, rather than through contact-rich physics. A real gripper would require force feedback, compliant control, and grasp quality assessment — none of which are present here.
- **No force, torque, or tactile sensing.** Grasp failures are invisible to the system. If the gripper misses the object or applies incorrect force, no sensor feedback exists to detect or recover from it.
- **Numerical IK only.** The system relies entirely on the KDL numerical solver. No closed-form analytical IK is implemented, which means convergence is not guaranteed near joint limits, singularities, or workspace boundaries. This is a known source of intermittent planning failures.
- **Static planning scene.** The MoveIt collision environment is fixed at startup. Active obstacles — a moving conveyor, a person entering the workspace, or any object outside the predefined scene — are entirely invisible to the planner.
- **Idealized QR perception pipeline.** The default mode uses a synthetic QR image stream rather than real camera input. Even in camera mode, Gazebo provides perfect lighting and zero lens distortion — conditions that would not hold in physical deployment, where lighting variation, motion blur, and partial occlusion routinely cause decoding failures.
- **Simplified collision form mismatch.** The MoveIt planning scene uses simplified box approximations for obstacles, while Gazebo renders more detailed visual geometry. This gap means that motions which appear safe in the planner can come closer to physical geometry than intended, and vice versa.
- **No fault recovery or cycle metrics.** The coordinator handles timeouts and basic fallbacks, but there is no structured fault recovery policy at the task level. Additionally, no performance data — planning time, success rate, or execution time per cycle — is logged or analyzed.

Taken together, these constraints are acceptable for a teaching and demonstration platform — the system successfully illustrates the full pick-scan-place workflow and validates the combination of all required ROS 2 components. They do, however, define a definite roadmap for future development, addressed in Section 18.

18. Recommended Future Improvements

Each limitation identified in Section 17 points directly toward a concrete improvement. The following extensions are ranked roughly by impact and implementation feasibility, starting with the changes that would most immediately close the gap between this demonstration platform and a deployable industrial system.

- **Perception-based object pose estimation.** Integrate a depth camera or point-cloud pipeline to estimate object pose at runtime, removing the dependency on fixed, pre-measured pick locations.
- **Contact-aware grasping.** Replace simulated object re-positioning with a Gazebo grasp plugin or contact-rich physics model, enabling realistic force and slip detection.
- **Analytical IK solver.** Derive or integrate a closed-form IK solution for this 6R arm with a spherical wrist. This would eliminate solver-timeout failures and significantly improve planning reliability near joint limits and in particular configurations.
- **Dynamic obstacle handling.** Update the MoveIt planning scene at runtime using sensor data, allowing the planner to react to moving obstacles, conveyor belts, or human operators entering the workspace.

- **Coordinator-level fault recovery.** Add structured retry and recovery policies at the task level so the system can handle repeated stage failures, missed grasps, or partial cycle completions without requiring a manual restart.
- **Planning scene fidelity improvement.** Tighten the alignment between MoveIt collision mesh and Gazebo visual geometry to within a few centimeters, diminishing the risk of planned paths passing closer to real obstacles than the planner anticipates.
- **Cycle performance logging.** Instrument the manipulation and coordinator nodes to record planning time, execution time, and success rate per stage. This data is key for both academic evaluation and field deployment decisions.
- **Conveyor and multi-object support.** Extend the workflow to handle a moving conveyor or a sequence of objects, bringing the simulation considerably closer to real industrial throughput requirements.

None of these extensions would require a fundamental redesign of the existing architecture. The clean package separation described in Section 3 means that individual modules — the IK solver, the attachment helper, the coordinator — can be replaced or upgraded independently without breaking the rest of the system.

19. Conclusion

This repository presents a coherent, well-structured industrial pick-scan-place cell that demonstrates how modern ROS 2 tooling can be assembled into a complete autonomous manipulation workflow. The separation of modeling, planning, perception, decision-making, simulation, and orchestration into distinct packages is not just good software hygiene — it is what makes the system comprehensible, testable, and extensible.

The robot itself is a clean 6R serial manipulator with a simplified 1-DOF gripper, modeled with sufficient kinematic and inertial detail for reliable MoveIt planning and Gazebo execution. From a kinematics standpoint, the project is particularly instructive because it contains a clearly derivable forward kinematic chain — readable directly from the URDF joint origins and axes — alongside a realistic inverse kinematics workflow based on the KDL numerical solver embedded in MoveIt 2.

That combination makes this workspace suitable not only for live demonstrations, but also for technical explanation, academic reporting, and oral defense preparation. The layered fallback strategies in the manipulation node, the clean data flow from perception to decision to action, and the explicit separation between the MoveIt planning scene and the Gazebo simulation environment are all patterns worth carrying forward into more advanced automation robotics projects.